
AGENDA 6

Estrutura de Decisão

SEAD- Superintendência de Ensino e Pesquisa nas
modalidades EaD e Aberta

GOVERNO DO ESTADO DE SÃO PAULO
EIXO TECNOLÓGICO DE INFORMAÇÃO E COMUNICAÇÃO
CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

DESENVOLVIMENTO DE SISTEMAS I

Expediente

Autores:

PAULO EDUARDO CARDOSO ANDRADE

TATIANE TOLENTINO DE ASSIS

Revisão Técnica:

Kelly Cristiane de Oliveira Dal Pozzo

São Paulo – SP, 2025



MOMENTO REFLEXÃO



Todos os dias, somos confrontados com escolhas que parecem simples, mas carregam consigo consequências importantes na nossa rotina e no modo como encaramos o mundo. O toque do despertador e o dilema entre levantar imediatamente ou ficar mais alguns minutos na cama é uma dessas decisões. Quando olhamos pela janela e vemos um céu nublado, ponderamos se vale a pena levar o guarda-chuva ou confiar que não irá chover. Optamos entre tomar o café em casa ou comprar algo pelo caminho, entre seguir a rota tradicional para o trabalho ou arriscar um novo trajeto para fugir do trânsito. Até mesmo dentro de um supermercado, decidimos se compramos um produto mais barato ou apostamos na marca de confiança. Essas pequenas escolhas, moldadas por condições externas e preferências pessoais, revelam um padrão: observamos informações, avaliamos opções e nos posicionamos. Essa lógica de tomada de decisão não se limita ao dia a dia. No ambiente profissional, especialmente no desenvolvimento de sistemas, ela se manifesta de forma ainda mais intensa. Imagine um analista de sistemas que precisa definir quais requisitos serão priorizados em um projeto. Ou um desenvolvedor que, diante de um erro inesperado, avalia as hipóteses antes de escolher uma abordagem para solucionar o problema.

Da mesma forma, quando ensinamos uma máquina a "pensar", estamos transferindo esse raciocínio humano. **Programar é traduzir condições e decisões para uma linguagem que o computador entende.** É como se codificássemos nossas escolhas, usando estruturas lógicas que permitem ao sistema reagir conforme o cenário apresentado. Por exemplo, um aplicativo de transporte verifica a localização do usuário para decidir qual motorista acionar; um sistema bancário avalia o saldo antes de liberar uma transação; um software de gestão define caminhos diferentes a partir das informações inseridas pelo usuário.

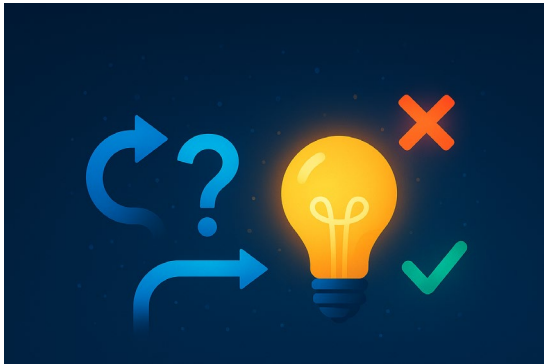
Quando nos damos conta de que programar é, essencialmente, decidir, percebemos que o processo vai além das linhas de código. Trata-se de compreender como as condições influenciam os resultados e como a lógica utilizada é que define o comportamento de sistemas. Quanto melhor entendemos o processo de tomada de decisões na vida real, mais apurada será nossa habilidade de criar sistemas eficientes, que respondem adequadamente aos desafios do cotidiano.

No fundo, construir algoritmos é treinar o computador para tomar decisões com base em critérios objetivos, transformando o raciocínio humano em lógica de programação. Assim como fazemos escolhas constantemente, os sistemas que desenvolvemos também precisam ser bem "ensinados" a escolher os melhores caminhos.

E se você pudesse ensinar uma máquina a tomar decisões, como escolheria quais critérios ela deve seguir?



O que torna uma decisão efetiva em um programa de computador?



Estrutura condicional é a ferramenta lógica que permite ao computador tomar decisões durante a execução de um programa. Ao invés de seguir sempre o mesmo caminho, o sistema pode escolher diferentes rotas dependendo de uma condição ser verdadeira ou falsa. Essa flexibilidade é fundamental na programação, pois torna os softwares capazes de reagir a situações variadas, criar interatividade, validar dados ou adaptar funções para o usuário.

Aprender sobre estruturas condicionais é essencial porque elas são a base do pensamento computacional: ao dominar esse conceito, você será capaz de criar programas que executam tarefas inteligentes, desde validar uma senha até decidir se determinado recurso deve ser disponibilizado. Sem as estruturas de decisão, um sistema não teria autonomia; toda lógica dependeria de ordens fixas, sem adaptação aos dados ou contexto.

No desenvolvimento de sistemas, essa habilidade é indispensável. Um profissional apto a elaborar algoritmos usando estruturas condicionais pode atender a requisitos reais do mercado, construir programas mais seguros e funcionais, e colaborar em equipes técnicas para projetos inovadores. Saber usar "if", "else" e "elif" é um passo inicial para evoluir no mundo da programação, resolver problemas práticos e abrir caminhos para desafios mais complexos, como automação e inteligência artificial.

Dominar estrutura condicional significa aprender como o computador analisa situações e toma decisões, preparando você para criar softwares dinâmicos, eficientes e alinhados às necessidades do mundo digital.

Assista o vídeo introdutório:



Disponível em: <https://www.youtube.com/watch?v=pTs8olKzrLM>
Acessado em: 19/03/2026

 PARA COMEÇAR O ASSUNTO

Em algum momento, todo programador percebe que escrever código vai além de simplesmente ordenar o computador a executar tarefas. É, na verdade, ensinar a máquina a tomar decisões e a selecionar caminhos diferentes conforme as condições encontradas durante o processo de execução. Até aqui, você já aprendeu a receber informações e mostrar resultados, mas o próximo passo transforma a programação em algo mais dinâmico e inteligente: e se o programa precisar escolher o que fazer?

Imagine um sistema de atendimento em uma empresa de tecnologia. Ao atender diferentes tipos de clientes, o programa precisa exibir mensagens específicas: saudar um usuário logado de maneira personalizada, oferecer descontos para compras acima de determinado valor, ou restringir acesso a funcionalidades conforme a idade informada. É nessas situações que o desvio condicional assume papel central — o coração da lógica de tomada de decisão. Com ele, o código passa a reagir a situações reais, como:

- Se a nota de um aluno for maior que 6 → o sistema confirma aprovação.
- Caso contrário → encaminha para a recuperação.
- Se o valor da compra for maior que R\$ 100,00 → aplica um desconto especial.
- Se a idade do cliente for inferior a 18 → restringe a operação por questões legais.

Essas decisões, embora simples à primeira vista, são responsáveis por garantir o funcionamento correto das regras de negócio em empresas de diferentes segmentos. O código deixa de ser uma sequência linear de comandos e passa a “pensar” como você, abrindo novas possibilidades e soluções para cada cenário encontrado.

Por exemplo, no mundo corporativo, um sistema de controle de acesso pode permitir a entrada apenas de funcionários cadastrados. Se um usuário não estiver registrado, o software envia uma mensagem de alerta e solicita validação. Em soluções logísticas, o programa pode definir rotas de entrega com base em condições específicas, otimizando recursos e reduzindo custos. Cada estrutura condicional abre caminhos para automatizar processos, tornar o sistema mais inteligente e prevenir erros que poderiam comprometer a operação ou a segurança dos dados.

Assim, aprender a implementar desvios condicionais é um passo estratégico para o profissional de desenvolvimento de sistemas: você estará capacitado a criar programas adaptáveis, capazes de interpretar as informações recebidas e escolher ações apropriadas conforme o contexto.

 MERGULHANDO NO TEMA

A estrutura condicional é um dos pilares da lógica de programação. Ela permite que o programa tome decisões automáticas, executando diferentes blocos de código conforme determinadas condições sejam verdadeiras ou falsas. Em termos simples, o computador analisa uma expressão lógica (uma comparação, um teste ou verificação) e, com base no resultado, escolhe qual caminho seguir na execução.

Essas estruturas são essenciais para que um programa não execute sempre a mesma sequência de instruções, mas consiga reagir ao que acontece durante sua execução, seja uma entrada do usuário, um cálculo

realizado, um sensor ou até mesmo um dado vindo de outro sistema.

No Python (assim como em diversas outras linguagens), essa decisão é feita com o comando `if`, que pode ser combinado com `elif` (“senão, se”) e `else` (“senão”) para ampliar as possibilidades de decisão.

Dessa forma, o código passa a lidar com diferentes situações, tornando-se mais dinâmico, inteligente e adaptável. É nesse momento que o programa deixa de apenas seguir instruções fixas e passa a responder ao contexto, avaliando condições e escolhendo ações de acordo com cada cenário.

Mas antes de conhecermos mais sobre estrutura de decisão, vamos compreender sobre um item fundamental para a estruturas, os operadores relacionais.

Operadores Relacionais

Os operadores relacionais são símbolos usados para comparar valores dentro de uma expressão lógica. Eles servem para verificar se um número é maior, menor, igual ou diferente de outro, e o resultado dessa comparação sempre será verdadeiro (True) ou falso (False).

Esses operadores são fundamentais na construção de estruturas condicionais, pois é a partir deles que o programa “decide” o que fazer. Por exemplo, ao comparar `idade >= 18`, o Python verifica se o valor da variável `idade` é maior ou igual a 18. Se for, o resultado será True; caso contrário, False.

De forma geral, os operadores relacionais permitem que o computador avalie situações e tome decisões com base em dados. Eles são usados para:

- Controlar fluxos de decisão (`if`, `elif`, `else`);
- Validar regras (como limites, permissões, faixas etárias, notas mínimas etc.);
- Comparar resultados de cálculos, sensores ou variáveis;
- Filtrar e classificar dados em programas mais complexos.

Os operadores relacionais são o que dão lógica às decisões do código, permitindo que ele analise, compare e reaja de maneira precisa às condições apresentadas. Veja a seguir a tabela dos operadores relacionais.

Operadores relacionais na linguagem Python

Operador	Lê-se	Exemplo	Resultado
<code>==</code>	igual a	<code>nota == 10</code>	Verdadeiro se nota for exatamente 10
<code>!=</code>	diferente de	<code>cidade != "SP"</code>	Verdadeiro se cidade não for "SP"
<code>></code>	maior que	<code>saldo > 0</code>	Verdadeiro se saldo for positivo
<code><</code>	menor que	<code>idade < 12</code>	Verdadeiro se idade menor que 12
<code>>=</code>	maior ou igual a	<code>idade >= 18</code>	Verdadeiro se idade for 18 ou mais
<code><=</code>	menor ou igual a	<code>media <= 6</code>	Verdadeiro se media for 6 ou menos



É importante destacar que, embora o conceito dos operadores relacionais seja o mesmo em diferentes linguagens de programação, a forma de escrevê-los pode variar. Por exemplo, enquanto o Python utiliza `==` para verificar igualdade, outras linguagens podem ter pequenas diferenças de sintaxe ou regras específicas. Por isso, ao aprender uma nova linguagem, é fundamental conhecer como esses operadores são representados e utilizados naquele contexto.

Entendendo a estrutura condicional

A estrutura condicional segue uma lógica de decisão simples e direta:

1. O programa avalia uma condição (algo que pode ser verdadeiro ou falso).
2. Se a condição for verdadeira, ele executa um bloco de comandos.
3. Caso contrário, pode executar um bloco alternativo.

Esse processo representa o funcionamento básico da tomada de decisão em um programa. A cada verificação, o sistema analisa a condição, escolhe um caminho e executa a ação correspondente. Tal comportamento também pode ser compreendido dentro do modelo de entrada, processamento e saída, muito utilizado na lógica de programação. A condição geralmente depende de uma entrada, que pode ser um valor digitado pelo usuário, um dado já armazenado ou uma informação recebida de outro sistema. Em seguida, o programa realiza o processamento, que é a análise dessa condição por meio de uma expressão lógica. Por fim, com base no resultado dessa análise, ocorre a saída, que corresponde à ação executada — como exibir uma mensagem, realizar um cálculo ou tomar uma decisão no fluxo do programa.

O computador não “pensa” como nós, mas segue uma lógica precisa e objetiva: tudo se baseia em valores booleanos, ou seja, verdadeiro (True) ou falso (False). É a partir dessa avaliação que o fluxo do programa é definido.

Vamos visualizar nosso exemplo na linguagem Python:



python

```
idade = int(input("Digite sua idade: "))  
if idade >= 18:  
    print("Você é maior de idade.")  
else:  
    print("Você ainda é menor de idade.")
```

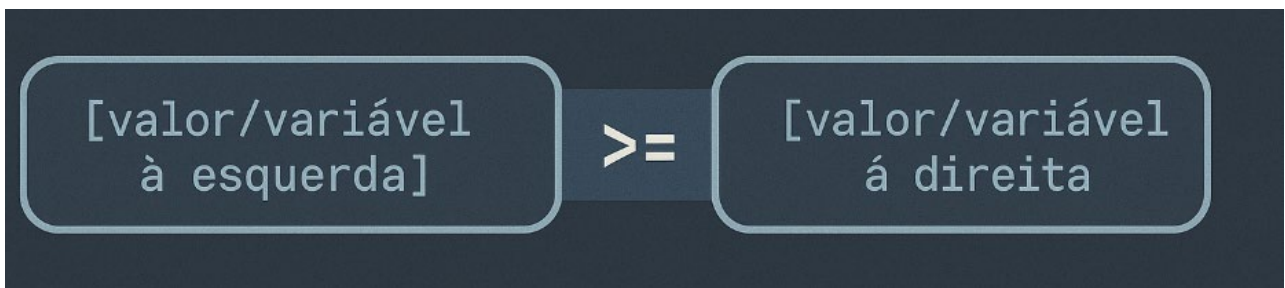
Compreendo o que acontece:

1. O programa pede a idade do usuário.
2. A linha `if idade >= 18:` verifica a condição:
3. Se for verdadeira, executa o bloco abaixo (mostra que é maior de idade).
4. Se for falsa, o `else` assume o controle e executa outro bloco (menor de idade).

Esse simples mecanismo permite que um código responda de maneiras diferentes a cada situação, e é isso que torna o programa interativo, capaz de tomar decisões.

Como montar a decisão do if (a “pergunta”)

Pense no `if` como uma pergunta que só pode ser respondida com Verdadeiro (True) ou Falso (False). Para formular essa pergunta, você combina valores ou variáveis com operadores de comparação. Esses operadores permitem verificar relações entre os dados, como igualdade, diferença ou maior e menor.



É justamente essa expressão que será analisada pelo `if`. Se o resultado for verdadeiro, o bloco de código associado será executado; caso contrário, o programa seguirá por outro caminho.

E como fica o comando na linguagem de programação Python:

python

```
if idade >= 18:
```

Passo a passo para montar uma boa condição

1



Defina o que quer decidir
(ex.: "Pode entrar?", "Tem desconto?", "Atingiu o limite?")

2



Escolha a variável e o comparativo
(ex.: idade, saldo, nota, nivel_bateria...)

3



Selecione o operador apropriado
(ex.: >=, <=, ==, !=)

4



Escreva a comparação com clareza
(idade >= 18
media < 6)

Se a expressão for True, o bloco do if é executado; se for False, o fluxo segue para elif/else (se existirem).

A importância da indentação no Python

Antes vamos relembrar o que é indentação:

Indentação é o alinhamento do código utilizado para indicar que um conjunto de instruções pertence a uma determinada estrutura. Ela mostra, de forma clara, quais comandos estão dentro de um if, de um else, de um for ou de uma função.

No Python, isso não é apenas uma questão estética — é obrigatório.

A partir do momento em que começamos a utilizar estruturas como if e else, surge um novo desafio: entender o que está dentro de cada bloco de código.

Um bloco de código é um conjunto de instruções que são executadas juntas dentro de uma estrutura. Por exemplo, todos os comandos que estão dentro de um if só serão executados se a condição for verdadeira. Em outras linguagens, como C, Java ou JavaScript, utilizamos chaves { } para delimitar o início e o fim de um bloco. No Python, essa separação é feita de uma forma mais simples e visual pela indentação (recuo de espaço).

O espaçamento no início das linhas define a hierarquia lógica do programa, ou seja, determina quais instruções serão executadas dentro de cada bloco.

python

```
if idade >= 18:
    print("Pode entrar.")
else:
    print("Acesso negado.")
```

Observe que a linha `print("Pode entrar.")` está recuada em relação ao `if`. Esse recuo indica que essa instrução pertence ao bloco do `if`.

Da mesma forma, o `print` dentro do `else` também está recuada, pois faz parte do outro bloco. Se você remover esse recuo, o Python não entende onde o bloco termina e gera um erro como:

IndentationError: expected an indented block

Por que a indentação é importante?

- Código mais legível, pois mostra a estrutura lógica de forma visual.
- Mais organizado, facilitando a manutenção.
- Menos propenso a erros, já que o programador é possível visualizar claramente o que pertence



Dicas:

- recuo padrão é de 4 espaços (ou uma tecla Tab).
- Nunca misture espaços e tabulações no mesmo arquivo, isso pode causar erros invisíveis.
- Em editores como VS Code, você pode ativar a opção "mostrar espaços" para enxergar melhor a indentação

Tipos de estruturas condicionais

Agora que você já compreende como as decisões são construídas, utilizando comparações e operadores relacionais, é hora de colocar tudo isso em prática. Com esses fundamentos, conseguimos dar o próximo passo: entender os tipos de estruturas condicionais que o Python oferece e como aplicá-las corretamente no código.

Cada tipo de estrutura tem uma função específica:

- algumas são usadas quando há apenas uma condição a ser verificada,
- outras quando há duas ou mais possibilidades,
- e há ainda aquelas em que uma decisão leva a outra, formando uma cadeia lógica.

Nos próximos exemplos, você verá como o programa escolhe diferentes caminhos conforme o resultado das condições, tornando-se capaz de analisar, reagir e decidir de forma cada vez mais autônoma e inteligente.

**Estrutura Condicional Simples**

A estrutura simples é usada quando existe apenas uma condição a ser verificada, e uma única ação deve ser executada caso ela seja verdadeira.



“Se isso acontecer, faça aquilo.”

Veja um exemplo em Python:

python

```
temperatura = float(input("Digite a temperatura: "))  
  
if temperatura > 30:  
    print("Está muito quente hoje!")
```

O programa verifica a condição ($\text{temperatura} > 30$). Se essa condição for verdadeira (True), ele executa o comando que está dentro do if. Caso seja falsa (False), o programa simplesmente continua sua execução normalmente, sem executar nenhuma ação adicional.

Esse tipo de estrutura é muito utilizado em situações simples, como exibir mensagens de alerta, realizar validações básicas ou apresentar avisos ao usuário.



Estrutura Condicional Composta

A estrutura composta é utilizada quando há duas possibilidades: uma ação para o caso da condição ser verdadeira e outra para o caso de ser falsa. Nesse tipo de estrutura, o programa sempre executa uma das duas alternativas, dependendo do resultado da condição. Se a condição for verdadeira, um bloco de código será executado; caso contrário, outro bloco será executado.

Essa lógica pode ser entendida como: “Se isso acontecer, faça isso; caso contrário, faça aquilo.”

Veja o exemplo em Python:

python

```
valor_compra = float(input("Digite o valor da compra: R$ "))

if valor_compra >= 100:
    print("Você ganhou um desconto de 10%!")
else:
    print("Sem desconto. Aproveite as próximas promoções!")
```

A condição (`valor_compra >= 100`) é avaliada pelo programa. Se for verdadeira, ele exibe a mensagem informando que o cliente tem direito ao desconto. Caso seja falsa, o bloco do `else` é executado, informando que não há desconto disponível.

Esse tipo de estrutura é bastante comum em situações como sistemas de promoções, definição de valores mínimos para frete grátis ou concessão de bonificações em compras.



Estrutura Condicional Encadeada (ou Aninhada)

A estrutura de decisão aninhada é utilizada quando há várias condições possíveis, e o programa precisa testá-las em sequência até encontrar uma que seja verdadeira.

Para isso, utiliza-se o comando `elif` (abreviação de “else if”), que permite adicionar novas verificações após o `if`, ampliando as possibilidades de decisão dentro do código.

Na prática é como dizer:



“Se for isso, faça isso. Senão, se for aquilo, faça aquilo. Caso contrário, faça outra coisa.”

Exemplo em Python:

python

```
nota = float(input("Digite sua nota final: "))

if nota >= 9:
    print("Excelente desempenho!")
elif nota >= 7:
    print("Aprovado!")
elif nota >= 5:
    print("Recuperação.")
else:
    print("Reprovado.")
```

O programa verifica cada condição em sequência. Assim que uma delas é avaliada como verdadeira, o bloco correspondente é executado e as demais condições são ignoradas. Caso nenhuma das condições anteriores seja atendida, o else final é responsável por tratar todas as demais situações.

Esse tipo de estrutura é bastante utilizado em cenários como classificação de desempenho, definição de faixas de pontuação e categorização de avaliações.



O Python oferece duas formas de criar decisões encadeadas:

1. Usando elif (forma mais direta e legível).
2. Ou aninhando um if dentro de um else (forma mais detalhada, mas menos limpa).

Essa segunda opção é chamada de estrutura condicional aninhada, pois um if é “aninhado” dentro do bloco de outro if ou else, formando uma espécie de hierarquia de decisões.



python

```

nota = float(input("Digite sua nota: "))

if nota >= 9:
    print("?? Excelente desempenho!")
else:
    if nota >= 7:
        print("Aprovado!")
    else:
        if nota >= 5:
            print("Recuperação.")
        else:
            print("Reprovado.")

```

Quando utilizar?

- elif → ideal para comparações paralelas, em uma mesma linha de decisão.
- else + if → útil para decisões dependentes, quando cada “nível” de decisão tem um contexto próprio.

Mini-checklist para testar estruturas condicionais

Antes de finalizar o código, é essencial verificar se suas condições estão realmente fazendo o que você espera. Um pequeno erro em um comparativo pode mudar completamente o comportamento do programa.

Por isso, revise cada decisão com atenção, seguindo este checklist:

1. A pergunta retorna True/False claramente?

Certifique-se de que sua condição sempre gera um resultado lógico. Uma boa condição é como uma pergunta direta, a resposta precisa ser “sim” ou “não”.

Exemplo: idade >= 18 é claro; já idade + 18 não faz sentido como teste lógico.

2. Os limites ($\geq$, $\leq$) estão corretos?

Pequenos detalhes, como usar > no lugar de >=, podem alterar o resultado de forma sutil. Pense em casos de fronteira (“limite”) e verifique se o programa inclui ou exclui o valor certo.

Exemplo: se o limite de aprovação é 6, >= 6 inclui o aluno que tirou 6, mas > 6 o reprova injustamente.

3. Os tipos estão certos (string, int, float)?

O Python diferencia tipos de dados. Comparar "10" com 10 não funciona, porque um é texto e o outro é número. Sempre converta as entradas para o tipo correto antes de comparar, por exemplo:

```
idade = int(input("Digite sua idade: "))
```



VOCÊ NO COMANDO

Você foi encarregado de criar um pequeno programa para o laboratório da Etec, que monitora a temperatura ambiente. O sistema precisa analisar a temperatura e emitir avisos diferentes conforme o valor lido.

Objetivo

Criar um programa em Python que receba a temperatura atual e, com base nela, mostre mensagens específicas, utilizando desvios condicionais aninhados.

Regras

1. Se a temperatura for menor que 0, exibir:
Temperatura abaixo de zero! Cuidado com o congelamento.
2. Se for menor que 15, exibir:
Frio intenso no laboratório.
3. Se for menor que 25, exibir:
Temperatura agradável.
4. Caso contrário:
Temperatura alta! Ligar sistema de refrigeração.

Passo a passo

1. Entrada de dados: Solicite a temperatura (float) com `input()`.
2. Primeira verificação: Cheque se é menor que zero.
Se for, mostre o aviso e encerre a decisão.
3. Caso contrário (else), entre na segunda verificação: Verifique se é menor que 15.
Se não, siga para a próxima camada (else dentro de else).
4. Continue testando dentro dos blocos até cobrir todas as possibilidades

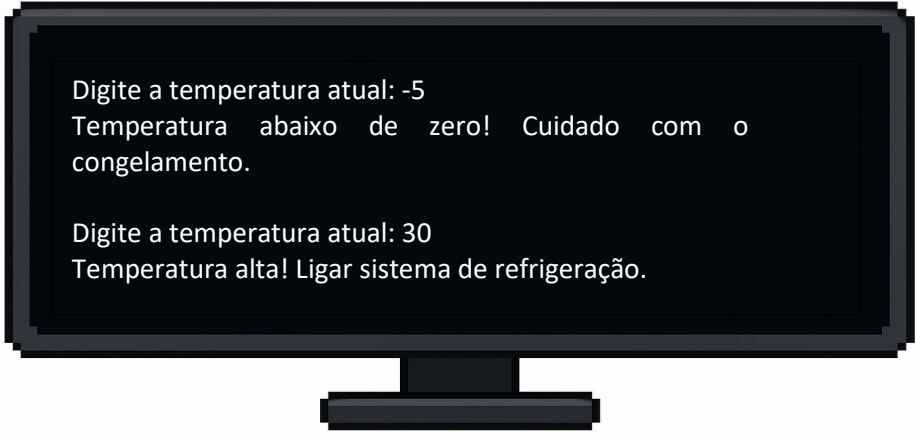
Veja se você acertou:

python

```
temperatura = float(input("Digite a temperatura atual: "))

if temperatura < 0:
    print("Temperatura abaixo de zero! Cuidado com o congelamento.")
else:
    if temperatura < 15:
        print("Frio intenso no laboratório.")
    else:
        if temperatura < 25:
            print("Temperatura agradável.")
        else:
            print("Temperatura alta! Ligar sistema de refrigeração.")
```

Teste de Execução



Digite a temperatura atual: -5
Temperatura abaixo de zero! Cuidado com o congelamento.

Digite a temperatura atual: 30
Temperatura alta! Ligar sistema de refrigeração.



Disponível em: https://www.youtube.com/watch?v=JDSanvM_V98
Acessado em 13/03/2026.